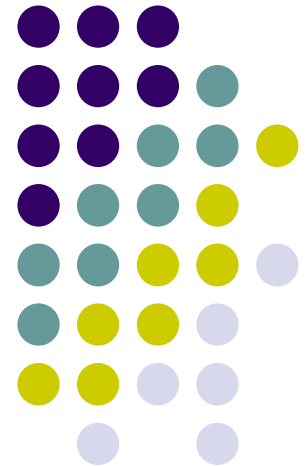
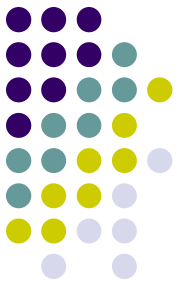


第4章 贪心算法

Do you remember the
story of monkey picking
maize?





一个小问题

- 有4种硬币，面值分别为25分，5分和1分
- 要找给顾客63分钱
- 求最少的硬币数？

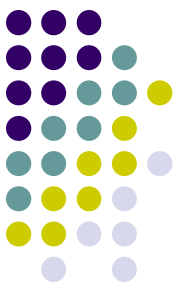
贪心

- $63 = 25 * 2 + 5 * 2 + 1 * 3$
- 如果面值改为1分，5分和11分
- 要找给顾客15分？
- $15 = 5 * 3$



当一个问题具有最优子结构性质时，可用动态规划法求解，但有时用贪心算法求解会更加的简单有效。

顾名思义，贪心算法总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的**局部最优**选择。当然，希望贪心算法得到的最终结果也是整体最优的。虽然贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。如单源最短路径问题，最小生成树问题等。在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是最优解的很好近似。



4.1 活动安排问题

活动安排问题就是要在所给的活动集合中选出最大的相容活动子集合，是可以用贪心算法有效求解的很好例子。该问题要求高效地安排一系列争用某一公共资源的活动。贪心算法提供了一个简单、漂亮的方法使得尽可能多的活动能兼容地使用公共资源。

思考一下，动态规划怎么做？



4.1 活动安排问题

设有 n 个活动的集合 $E = \{1, 2, \dots, n\}$ ，其中每个活动都要求使用同一资源，如演讲会场等，而在同一时间内只有一个活动能使用这一资源。每个活动 i 都有一个要求使用该资源的起始时间 s_i 和一个结束时间 f_i ，且 $s_i < f_i$ 。如果选择了活动 i ，则它在全开时间区间 $[s_i, f_i)$ 内占用资源。若区间 $[s_i, f_i)$ 与区间 $[s_j, f_j)$ 不相交，则称活动 i 与活动 j 是相容的。也就是说，当 $s_i \geq f_j$ 或 $s_j \geq f_i$ 时，活动 i 与活动 j 相容。

活动安排问题就是要在所给的活动集合中选出最大的相容活动子集合。



4.1 活动安排问题

(用动态规划方法)

- 步骤1: 分析最优解的结构特征

— 构造子问题空间:

$$S_{ij} = \{ a_k \in S : f_i \leq s_k < f_k \leq s_j \}$$

S_{ij} 包含了所有与 a_i 和 a_j 相兼容的活动, 并且与不迟于 a_i 结束和不早于 a_j 开始的活动兼容。此外, 虚构活动 a_0 和 a_{n+1} , 其中 $f_0 = 0, s_{n+1} = \infty$ 。原问题即为寻找 $S_{0,n+1}$ 中最大兼容活动子集。



4.1 活动安排问题

● 步骤2: 递归地定义最优解的值

设 $c[i, j]$ 为 S_{ij} 中最大兼容子集中的活动数。当 $S_{ij} = \phi$ 时, $c[i, j] = 0$ 。对于一个非空子集 S_{ij} , 如果 a_k 在 S_{ij} 的最大兼容子集中被使用, 则子问题 S_{ik} 和 S_{kj} 的最大兼容子集也被使用。从而:

$$c[i, j] = c[i, k] + c[k, j] + 1$$

由于 S_{ij} 的最大子集一定使用了 i 到 j 中的某个值 k , 通过检查所有可能的 k 值, 就可以找到最好的一个。因此, $c[i, j]$ 的完整递归定义为:

$$c[i, j] = \begin{cases} 0 & S_{ij} = \phi \\ \max_{i < k < j} \{c[i, k] + c[k, j] + 1\} & S_{ij} \neq \phi \end{cases}$$

问题: k 有 $j-i-1$ 种选择, 每种选择会导致2个完全不同的子问题产生, 因此, 动态规划算法的计算量比较大!!! 一个直观想法是直接选择 k 的值, 使得一个子问题为空, 从而加快计算速度! 这就导致了贪心算法!



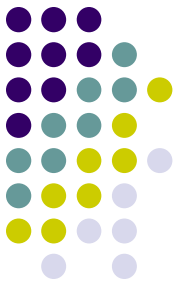
4.1 活动安排问题

(用贪心算法)

- **贪心策略**：对输入的活动以其完成时间的**非减序**排列，算法每次总是选择**具有最早完成时间**的相容活动加入最优解集中。直观上，按这种方法选择相容活动为未安排活动留下尽可能多的时间。也就是说，该算法的贪心选择的意义是**使剩余的可安排时间段极大化**，以便安排尽可能多的相容活动。

例：设待安排的11个活动的开始时间和结束时间按结束时间的非减序排列如下：

i	1	2	3	4	5	6	7	8	9	10	11
S[i]	1	3	0	5	3	5	6	8	8	2	12
f[i]	4	5	6	7	8	9	10	11	12	13	14

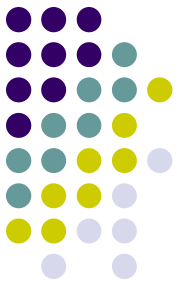


4.1 活动安排问题

下面给出解活动安排问题的贪心算法GreedySelector :

```
template<class Type>
void GreedySelector(int n, Type s[], Type f[], bool A[])
{
    A[1]=true;
    int j=1;
    for (int i=2;i<=n;i++) {
        if (s[i]>=f[j]) { A[i]=true; j=i; }
        else A[i]=false;
    }
}
```

各活动的起始时间和结束时间存储于数组s和f中且按结束时间的非减序排列

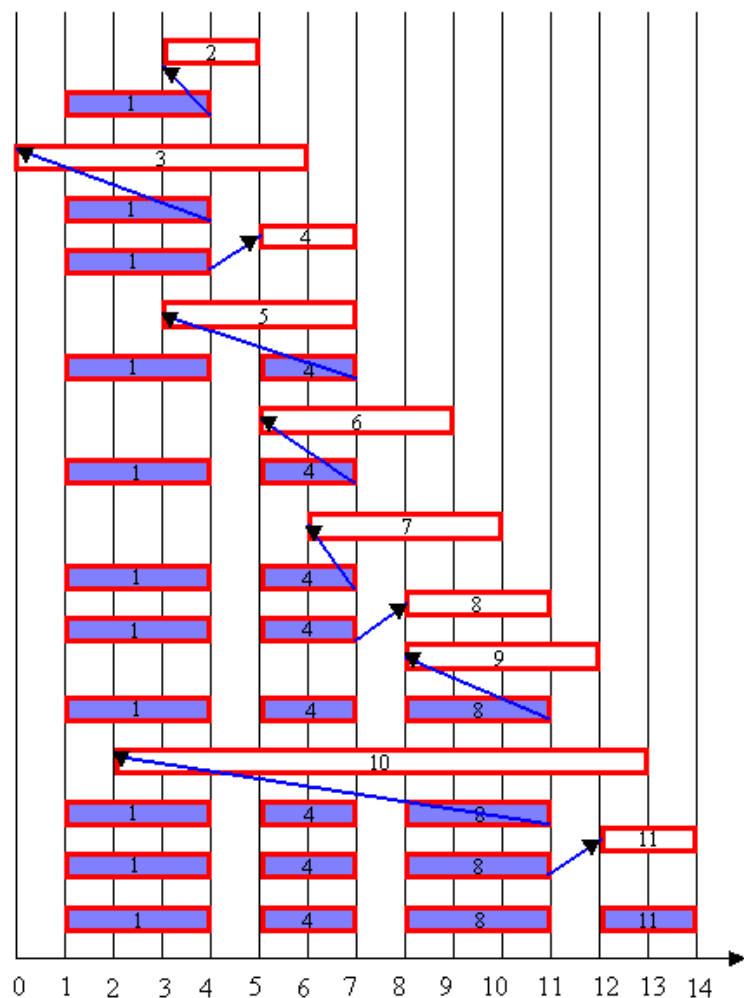


4.1 活动安排问题

例： 设待安排的11个活动的开始时间和结束时间按结束时间的非减序排列如下：

i	1	2	3	4	5	6	7	8	9	10	11
S[i]	1	3	0	5	3	5	6	8	8	2	12
f[i]	4	5	6	7	8	9	10	11	12	13	14

4.1 活动安排问题



算法greedySelector 的计算过程如左图所示。图中每行相应于算法的一次迭代。阴影长条表示的活动是已选入集合A的活动，而空白长条表示的活动是当前正在检查相容性的活动。



11个活动已按结束时间排序，用贪心算法求解：

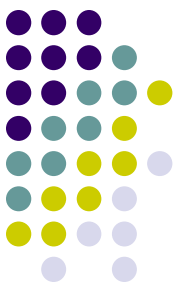
i	1	2	3	4	5	6	7	8	9	10	11
start_time _i	1	3	0	5	3	5	6	8	8	2	12
finish_time _i	4	5	6	7	8	9	10	11	12	13	14

time	a ₁	a ₂	a ₃	a ₄	a ₅	a ₆	a ₇	a ₈	a ₉	a ₁₀	a ₁₁
0											
1											
2											
3											
4											
5											
6											
7											
8											
9											
10											
11											
12											
13											
14											

相容活动：{a₃, a₉, a₁₁},
 {a₁, a₄, a₈, a₁₁}, {a₂, a₄, a₉, a₁₁}

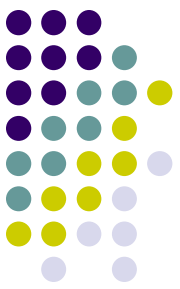


time	a ₁	a ₂	a ₃	a ₄	a ₅	a ₆	a ₇	a ₈	a ₉	a ₁₀	a ₁₁
0											
1											
2											
3											
4											
5											
6											
7											
8											
9											
10											
11											
12											
13											
14											



4.1 活动安排问题

算法greedySelector的效率极高。当输入的活动已按结束时间的非减序排列，算法只需 $O(n)$ 的时间安排 n 个活动，使最多的活动能相容地使用公共资源。如果所给出的活动未按非减序排列，可以用 $O(n\log n)$ 的时间重排。



4.1 活动安排问题

若被检查的活动 i 的开始时间 S_i 小于最近选择的
活动 j 的结束时间 f_j ，则不选择活动 i ，否则选择
活动 i 加入集合 A 中。

贪心算法并不总能求得问题的整体最优解。但
对于活动安排问题，贪心算法greedySelector却
总能求得的整体最优解，即它最终所确定的相容
活动集合 A 的规模最大。这个结论可以用数学归纳
法证明。



● 贪心算法的正确性证明：

定理16.1： 对于任意非空子问题 S_{ij} ，设 a_m 是 S_{ij} 中具有最早结束时间的活动，即 $f_m = \min\{f_k : a_k \in S_{ij}\}$ ，则：

- 1) 活动 a_m 在 S_{ij} 的某最大兼容活动子集中被使用；
- 2) 子问题 S_{im} 为空，所以选择 a_m 将使子问题 S_{mj} 为唯一可能非空的子问题。

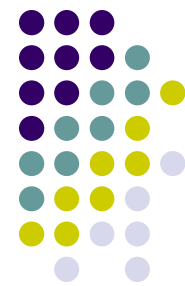


4.2 贪心算法的基本要素

本节着重讨论可以用贪心算法求解的问题的一般特征。

对于一个具体的问题，怎么知道是否可用贪心算法解此问题，以及能否得到问题的最优解呢？这个问题很难给予肯定的回答。

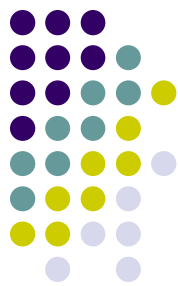
但是，从许多可以用贪心算法求解的问题中看到这类问题一般具有2个重要的性质：**贪心选择性质**和**最优子结构性**。



4.2 贪心算法的基本要素

基本思想：

- 从问题的某一个初始解出发，通过一系列的贪心选择——当前状态下的局部最优选择，逐步逼近给定的目标，尽可能快地求得更好的解。
- 在贪心算法(greedy method)中采用逐步构造最优解的方法。在每个阶段，都作出一个按某个评价函数最优的决策，该评价函数最优称为贪心准则(greedy criterion)。
- 贪心算法的正确性，就是要证明按贪心准则求得的解是全局最优解。



4.2 贪心算法的基本要素

基本步骤：

- ① 决定问题的最优子结构；
- ② 设计出一个递归解；
- ③ 证明在递归的任一阶段，最优选择之一总是贪心选择。那么，做贪心选择总是安全的。
- ④ 证明通过做贪心选择，所有子问题（除一个以外）都为空。
- ⑤ 设计出一个实现贪心策略的递归算法。
- ⑥ 将递归算法转换成迭代算法。



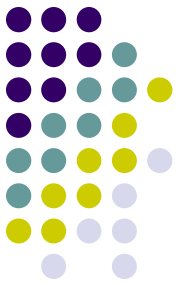
4.2 贪心算法的基本要素

1、贪心选择性质

所谓**贪心选择性质**是指所求问题的**整体最优解**可以通过一系列**局部最优**的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。

动态规划算法通常以**自底向上**的方式解各子问题，而贪心算法则通常以**自顶向下**的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。

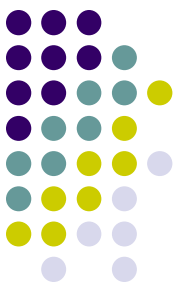
对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。



4.2 贪心算法的基本要素

2、最优子结构性质

当一个问题最优解包含其子问题的最优解时，称此问题具有**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。



3、贪心算法与动态规划算法的差异

- 贪心算法和动态规划算法都要求问题具有最优子结构性质，但是，两者存在着巨大的差别。

Dynamic Programming

- At each step, the choice is determined based on solutions of subproblems.
- Sub-problems are solved first.
- Bottom-up approach
- Can be slower, more complex

Greedy Algorithms

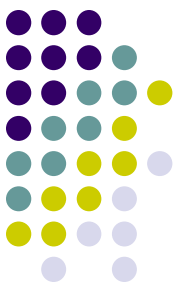
- At each step, we quickly make a choice that currently looks best.
-A local optimal (greedy) choice.
- Greedy choice can be made first before solving further sub-problems.
- Top-down approach
- Usually faster, simpler



4.2 贪心算法的基本要素

3、贪心算法与动态规划算法的差异

贪心算法和动态规划算法都要求问题具有最优子结构性质，这是2类算法的一个共同点。但是，对于具有**最优子结构**的问题应该选用贪心算法还是动态规划算法求解？是否能用动态规划算法求解的问题也能用贪心算法求解？下面研究2个经典的**组合优化问题**，并以此说明贪心算法与动态规划算法的主要差别。

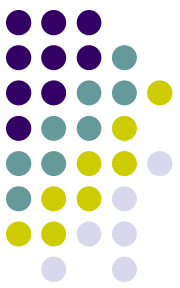


4.2 贪心算法的基本要素

- 0-1背包问题:

给定 n 种物品和一个背包。物品 i 的重量是 W_i ，其价值为 V_i ，背包的容量为 C 。应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

在选择装入背包的物品时，对每种物品 i 只有2种选择，即装入背包或不装入背包。不能将物品 i 装入背包多次，也不能只装入部分的物品 i 。



4.2 贪心算法的基本要素

- 背包问题：

与0-1背包问题类似，所不同的是在选择物品 i 装入背包时，可以选择物品 i 的一部分，而不一定要全部装入背包， $1 \leq i \leq n$ 。

这2类问题都具有最优子结构性质，极为相似，但背包问题可以用贪心算法求解，而0-1背包问题却不能用贪心算法求解。



例

●例：若 $n = 3$, $w = (10, 20, 30)$, $v = (60, 100, 120)$, $c = 50$, 则

➤对于0-1背包问题，可行解为：

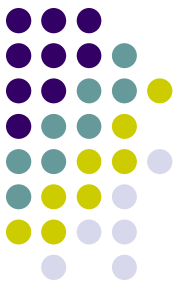
$$(x_1, x_2, x_3) = (0, 1, 1) = 220$$

$$(x_1, x_2, x_3) = (1, 1, 0) = 160$$

$$(x_1, x_2, x_3) = (1, 0, 1) = 180$$

最优解为：选择物品2和物品3，总价值为220

➤对于小数背包问题，按照物品价值率最大的贪心选择策略，其解为(10, 20, 20)，总价值为240。



4.2 贪心算法的基本要素

用贪心算法解背包问题的基本步骤：

首先计算每种物品单位重量的价值 V_i/W_i ，然后，依贪心选择策略，将尽可能多的单位重量价值最高的物品装入背包。若将这种物品全部装入背包后，背包内的物品总重量未超过 C ，则选择单位重量价值次高的物品并尽可能多地装入背包。依此策略一直地进行下去，直到背包装满为止。

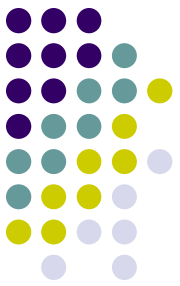


4.2 贪心算法的基本要素

对于0-1背包问题，贪心选择之所以不能得到最优解是因为在这种情况下，它无法保证最终能将背包装满，部分闲置的背包空间使每公斤背包空间的价值降低了。事实上，在考虑0-1背包问题时，应比较选择该物品和不选择该物品所导致的最终方案，然后再作出最好选择。由此就导出许多互相重叠的子问题。这正是该问题可用动态规划算法求解的另一重要特征。

实际上也是如此，动态规划算法的确可以有效地解0-1背包问题。

具体算法可描述如下页：



4.2 贪心算法的基本要素

```
void Knapsack(int n,float M,float v[],float
w[],float x[])
{
    Sort(n,v,w);
    int i;
    for (i=1;i<=n;i++) x[i]=0;
    float c=M;
    for (i=1;i<=n;i++) {
        if (w[i]>c) break;
        x[i]=1;
        c-=w[i];
    }
    if (i<=n) x[i]=c/w[i];
}
```

算法knapsack的主要计算时间在于将各种物品依其单位重量的价值从大到小排序。因此，算法的计算时间上界为 $O(n \log n)$ 。为了证明算法的正确性，还必须证明背包问题具有贪心选择性质。



- **问题描述**：给定 n 种物品和一个背包。物品 i 的重量是 W_i ，其价值为 V_i ，背包的容量为 C 。应如何选择装入背包的物品，使得装入背包中物品的总价值最大？这里，在选择物品 i 装入背包时，**可以选择物品 i 的一部分**，而不一定要全部装入背包。

$$\begin{aligned} \max & \sum_{i=1}^n v_i x_i && x_i \text{ 为装入物品 } i \text{ 的比例} \\ \begin{cases} \sum_{i=1}^n w_i x_i \leq c & w_i > 0 \\ 0 \leq x_i \leq 1 & i = 1, 2, \dots, n \\ v_i > 0, w_i > 0, c > 0 & i = 1, 2, \dots, n \end{cases} && \begin{aligned} & w_i \text{ 为重量, } c \text{ 为背包容量} \\ & v_i \text{ 为价值} \\ & v_i / w_i \text{ 为价值率(单位重量价值)} \end{aligned} \end{aligned}$$

例子： $n=3$, $c=20$, $v=(25, 24, 15)$, $w=(18, 15, 10)$ ，列举4个可行解：

	(x_1, x_2, x_3)	$\sum w_i x_i$	$\sum v_i x_i$
①	$(1/2, 1/3, \frac{1}{4})$	16.5	24.5
②	$(1, 2/15, 0)$	20	28.2
③	$(0, 2/3, 1)$	20	31
④	$(0, 1, \frac{1}{2})$	20	31.5 (最优解)



贪心策略设计:

- 策略1: 按价值最大贪心, 是目标函数增长最快。
按价值排序从高到低选择物品→②解(次最优)
- 策略2: 按重量最小贪心, 使背包增长最慢。
按重量排序从小到大选择物品→③解(次最优解)
- 策略3: 按价值率最大贪心, 使单位重量价值增长最快。
按价值率排序从大到小选择物品→④解(最优)



● 贪心选择的最优性证明

➤ 定理：如果 $v_1/w_1 \geq v_2/w_2 \geq \dots \geq v_n/w_n$ ，则 GreedyKnapsack 算法对于给定的背包问题实例生成一个最优解

➤ 证明基本思想：

把贪心解与任一最优解相比较，如果这两个解不同，就去找开始不同的第一个 x_i ，然后设法用贪心解的 x_i 去代换最优解的 x_i ，并证明最优解在分量代换之后其总价值保持不变，反复进行下去，直到新产生的最优解与贪心解完全一样，从而证明了贪心解是最优解。



- 证明: 设 $(x_1, \dots, x_n)$ 是贪心算法求得的解

Case1: 所有 $x_i = 1$ 。显然该解就是最优解。

Case2: 设 $X = (1, \dots, 1, x_j, 0, \dots, 0)$ $x_j \neq 1, 1 \leq j \leq n$ 。下证 X 就是最优解。设问题的最优解 $Y = (y_1, \dots, y_n)$, 则存在 k 使得 $y_k \neq x_k$ 的最小下标 (否则 $Y = X$, 得证)。

首先, 可以证明 $y_k < x_k$ 。(反证: 若 $y_k > x_k$, 则 $x_k \neq 1, \therefore k \geq j$

$$\Rightarrow \sum_{i=1}^n w_i y_i \geq \sum_{i=1}^k w_i y_i > \sum_{i=1}^k w_i x_i \geq \sum_{i=1}^j w_i x_i = c \therefore Y \text{ 不是可行解, 矛盾})$$

下面改造 Y 成为新解 $Z = (z_1, \dots, z_n)$, 并使 Z 仍为最优解。将 y_k 增加到 x_k , 从 $(y_{k+1}, \dots, y_n)$ 中减同样的重量使总量仍是 c 。即,

$$z_i = x_i \quad i = 1, 2, \dots, k; \quad \text{和} \quad w_k(z_k - y_k) = \sum_{i=k+1}^n w_i(y_i - z_i)$$



$$\begin{aligned}\therefore \sum_{i=1}^n w_i z_i &= \sum_{i=1}^{k-1} w_i z_i + w_k z_k + \sum_{i=k+1}^n w_i z_i \\&= \sum_{i=1}^{k-1} w_i y_i + w_k z_k + \left(\sum_{i=k+1}^n w_i y_i - w_k (z_k - y_k) \right) = \sum_{i=1}^n w_i y_i = c \\ \therefore \sum_{i=1}^n v_i z_i &= \sum_{i=1}^n v_i y_i + (z_k - y_k) v_k - \sum_{i=k+1}^n (y_i - z_i) v_i \quad // \because Z \text{ 由 } Y \text{ 变得的} \\&= \sum_{i=1}^n v_i y_i + (z_k - y_k) w_k v_k / w_k - \sum_{i=k+1}^n (y_i - z_i) w_i v_i / w_i \\&\geq \sum_{i=1}^n v_i y_i + \left[(z_k - y_k) w_k - \sum_{i=k+1}^n (y_i - z_i) w_i \right] v_k / w_k \quad // \text{利用 } v_i / w_i \downarrow \\&= \sum_{i=1}^n v_i y_i\end{aligned}$$

$\therefore Z$ 也是最优解, 且 $z_i = x_i \ i=1, \dots, k$; 重复上面过程 $\Rightarrow X$ 为最优解。



- 特殊的0-1背包问题:



找钱问题

● 问题定义:

使用2角5分, 1角, 5分和1分四种面值的硬币时 (各种硬币数量不限), 设计一个找A分钱的贪心算法, 并证明算法能产生一个最优解。设货币种类 $P=\{p_1, p_2, p_3, p_4\}$, d_i 和 x_i 分别是 p_i 的货币单位和选择数量, 问题的形式描述为:

$$\begin{aligned} & \min \left\{ \sum_{i=1}^4 x_i \right\} \\ \text{s.t. } & \begin{cases} \sum_{i=1}^4 d_i x_i = A \\ x_i \text{ 为非负整数} \quad 1 \leq i \leq 4 \end{cases} \end{aligned}$$



- 贪心策略

$$x_1 = \lfloor A/d_1 \rfloor$$

$$x_3 = \lfloor (A - d_1x_1 - d_2x_2)/d_3 \rfloor$$

$$x_2 = \lfloor (A - d_1x_1)/d_2 \rfloor$$

$$x_4 = A - d_1x_1 - d_2x_2 - d_3x_3$$

- 算法

GreedyChange(d[], x[], A)

{//输出x[1..4]}

 d[1]=25, d[2]=10, d[3]=5, d[4]=1;

 for i=1 to 3 do

 { x[i]=A/d[i]; A-=x[i]*d[i];

 }

 x[4]=A;

}



● 最优子结构性证明：

设 $X=(x_1, x_2, x_3, x_4)$ 是问题钱数为 A 的最优解，则 $X'=(0, x_2, x_3, x_4)$ 是子问题钱数为 $A-d_1x_1$ 的最优解。

下面反证：若不然， $Y=(0, y_2, y_3, y_4)$ 是子问题钱数为 $A-d_1x_1$ 的最优解，即 Y 优于 X'

$$\sum_{i=2}^4 y_i < \sum_{i=2}^4 x_i \quad \text{且} \quad \sum_{i=2}^4 d_i y_i = A - d_1 x_1$$

$$\Rightarrow x_1 + \sum_{i=2}^4 y_i < \sum_{i=1}^4 x_i \quad \text{且} \quad d_1 x_1 + \sum_{i=2}^4 d_i y_i = A$$

$\Rightarrow (x_1, y_2, y_3, y_4)$ 比 (x_1, x_2, x_3, x_4) 更优，矛盾。



● 贪心选择性质证明:

设 $X=(x_1, x_2, x_3, x_4)$ 是贪心解, $Y=(y_1, y_2, y_3, y_4)$ 是最优解.

可以证明: $x_1 = y_1$

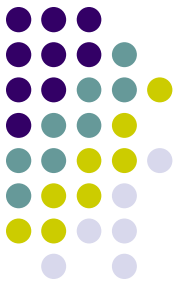
如果 $x_1 < y_1$, 由 $x_1 = \lfloor A/d_1 \rfloor \Rightarrow x_1 \leq A/d_1 < x_1 + 1 \Rightarrow d_1 x_1 \leq A <$

$d_1(x_1 + 1)$; 因此, $\sum_{i=1}^4 d_i y_i \geq d_1 y_1 \geq d_1(x_1 + 1) > A$, 产生矛盾;

如果 $x_1 > y_1$, 则 $10y_2 + 5y_3 + 1y_4 \geq 25$ (否则, $25y_1 + 10y_2 + 5y_3 + 1y_4 < 25(y_1 + 1) \leq 25x_1 + 10x_2 + 5x_3 + 1x_4 = A$, 矛盾), 因此用一个25分的硬币替代等值的低于25分的硬币若干个 (至少 ≥ 3), 于是 y 不是最优解;

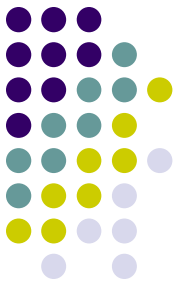
综上, 只能 $x_1 = y_1$; 类似可以继续证得 $x_2 = y_2$, $x_3 = y_3$, $x_4 = y_4$;

$\therefore X$ 是最优解



找钱问题

- 思考：如果硬币面值改为1分、5分和1角1分，要找给顾客的是1角5分，是否可以用贪心算法？



4.3 最优装载

有一批集装箱要装上一艘载重量为 c 的轮船。其中集装箱 i 的重量为 w_i 。最优装载问题要求确定在装载体积不受限制的情况下，将尽可能多的集装箱装上轮船。

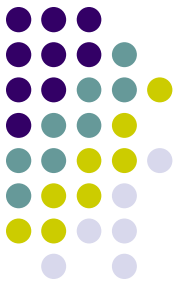
形式化描述如下：

$$\begin{aligned} & \max \sum_{i=1}^n x_i \\ & s.t. \begin{cases} \sum_{i=1}^n w_i x_i \leq c & w_i > 0 \\ x_i \in \{0,1\} & i = 1, 2, \dots, n \end{cases} \end{aligned}$$



4.3 最优装载

- **贪心策略：**从剩下的货箱中，选择重量最小的货箱。这种选择次序可以保证所选的货箱总重量最小从而可以装载更多的货箱。根据这种贪心策略，首先选择最轻的货箱，然后选择次轻的货箱，如此下去直到所有货箱均装上船或者船上不能容纳其他任何一个货箱。
- **计算实例：**假设 $n = 8$, $[w_1, w_2, \dots, w_8] = [100, 200, 50, 90, 150, 50, 20, 80]$, $c = 400$ 。
利用贪心算法时，所考察货箱的顺序为7, 3, 6, 8, 4, 1, 5, 2。
货箱7, 3, 6, 8, 4, 1的总重量为390个单位且已被装载，剩下的装载能力为10个单位，小于剩下的任何一个货箱。
在这种贪心解决算法中得到 $[x_1, x_2, \dots, x_8] = [1, 0, 1, 1, 0, 1, 1, 1]$ ，且 $\sum x_i = 6$



4.3 最优装载

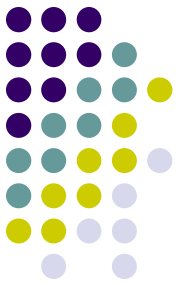
```
template<class Type>
void Loading(int x[], Type w[], Type c, int n)
{
    int *t = new int [n+1];

    Sort(w, t, n);

    for (int i = 1; i <= n; i++) x[i] = 0;

    for (int i = 1; i <= n && w[t[i]] <= c; i++) {x[t[i]] = 1; c -= w[t[i]];}

}
```



4.3 最优装载

2、贪心选择性质

可以证明最优装载问题具有贪心选择性质。

3、最优子结构性质

最优装载问题具有最优子结构性质。

由最优装载问题的贪心选择性质和最优子结构性质，容易证明算法loading的正确性。

算法loading的主要计算量在于将集装箱依其重量从小到大排序，故算法所需的计算时间为 $O(n \log n)$ 。



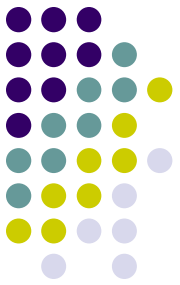
4.4 哈夫曼编码

哈夫曼编码是广泛地用于数据文件压缩的十分有效的编码方法。其压缩率通常在20%~90%之间。哈夫曼编码算法用字符在文件中出现的频率表来建立一个用0, 1串表示各字符的最优表示方式。

给出现频率高的字符较短的编码，出现频率较低的字符以较长的编码，可以大大缩短总码长。

1、前缀码

对每一个字符规定一个0, 1串作为其代码，并要求任一字符的代码都不是其它字符代码的前缀。这种编码称为前缀码。

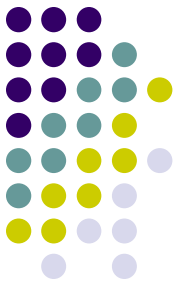


一个例子

- A character-coding problem. A data file of 100,000 characters contains only the characters a–f, with the frequencies:

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
Frequency	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

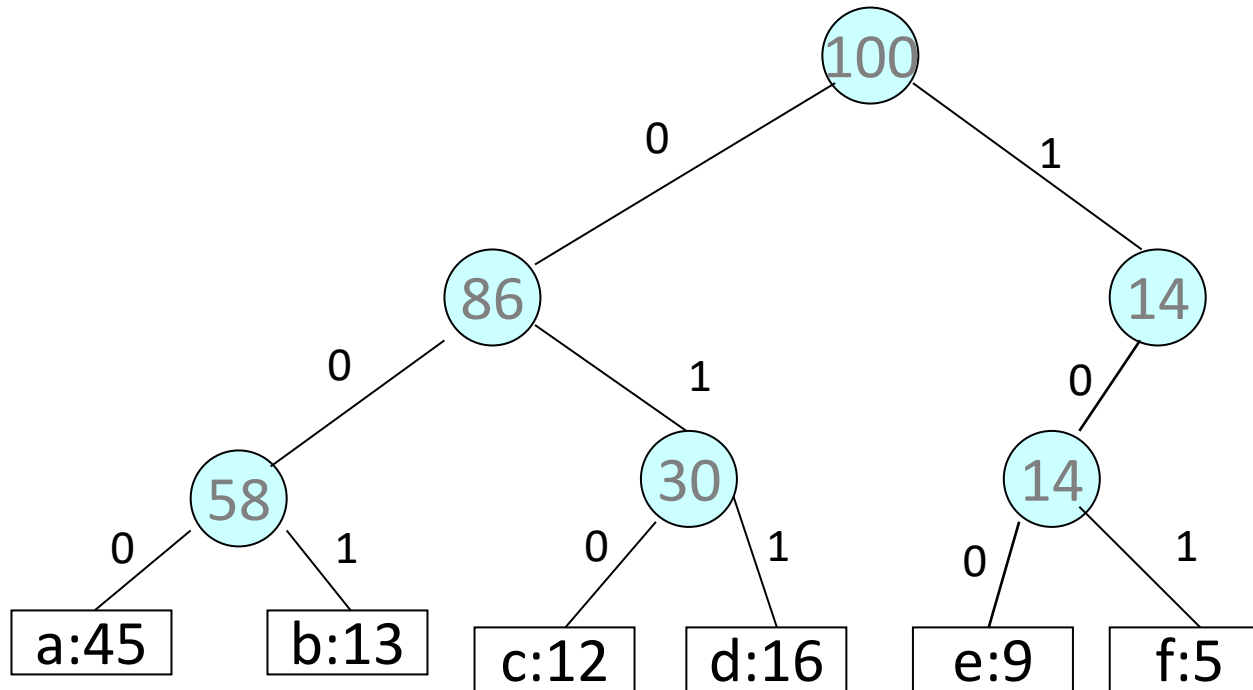
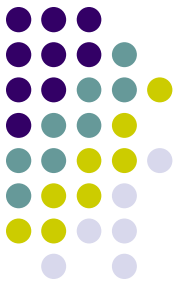
- How much do we save in this case?
 - **Fixed-length code:** $100,000 \times 3 = 300,000$
 - **Variable-length code:** $(45 \times 1 + 13 \times 3 + 12 \times 3 + 16 \times 3 + 9 \times 4 + 5 \times 4) \times 1,000 = 224,000$ bits



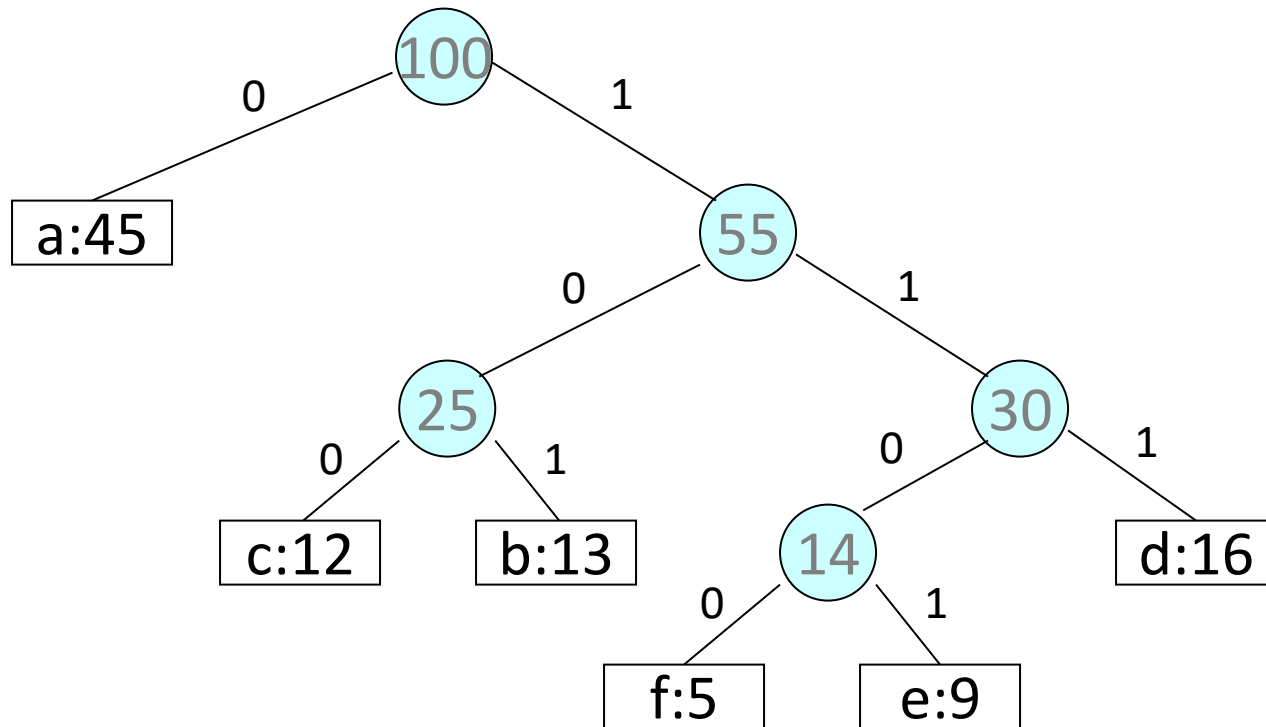
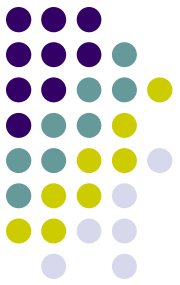
前缀码

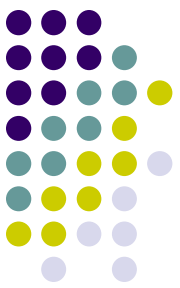
- **Prefix codes**: no codeword is also a prefix of some other codeword.
 - It can be shown that optimal coding can be done with prefix code
- **Encoding**: concatenate the codewords representing each character of the file.
 $abc \rightarrow 0 \cdot 101 \cdot 100 \rightarrow 0101100$.
- Prefix codes are desirable because they simplify **decoding**.
 $001011101 \rightarrow 0 \cdot 0 \cdot 101 \cdot 1101 \rightarrow aabe$.

The optimal prefix code



The optimal prefix code





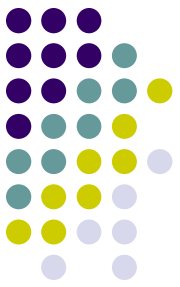
4.4 哈夫曼编码

编码的前缀性质可以使译码方法非常简单。

表示最优前缀码的二叉树总是一棵完全二叉树，即树中任一结点都有2个儿子结点。

平均码长定义为：
$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

使平均码长达到最小的前缀码编码方案称为给定编码字符集C的最优前缀码。



4.4 哈夫曼编码

2、构造哈夫曼编码

哈夫曼提出构造最优前缀码的贪心算法，由此产生的编码方案称为哈夫曼编码。

哈夫曼算法以自底向上的方式构造表示最优前缀码的二叉树T。

算法以 $|C|$ 个叶结点开始，执行 $|C|-1$ 次的“合并”运算后产生最终所要求的树T。

构造哈夫曼编码

f:5

e:9

c:12

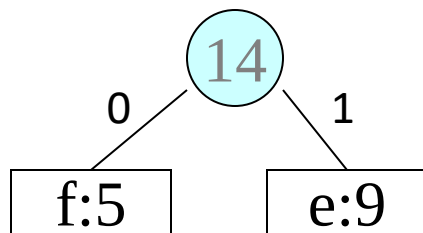
b:13

d:16

a:45



构造哈夫曼编码



c:12

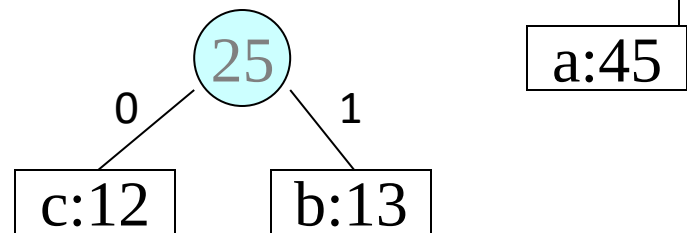
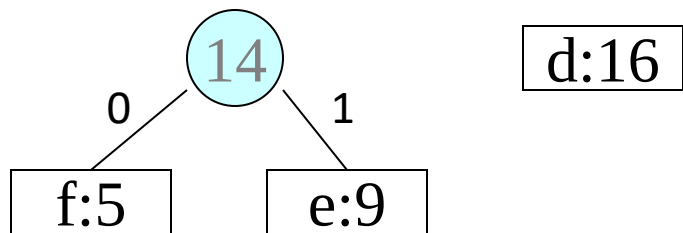
b:13

d:16

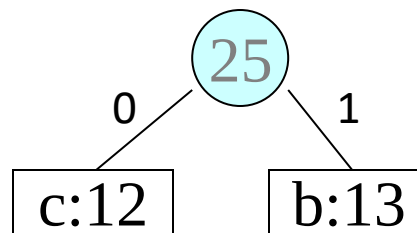
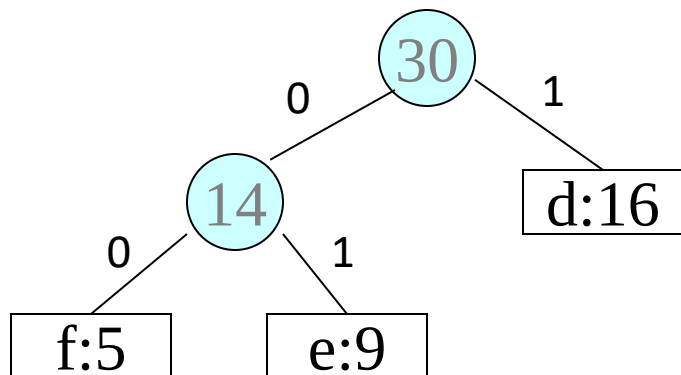
a:45



构造哈夫曼编码



构造哈夫曼编码



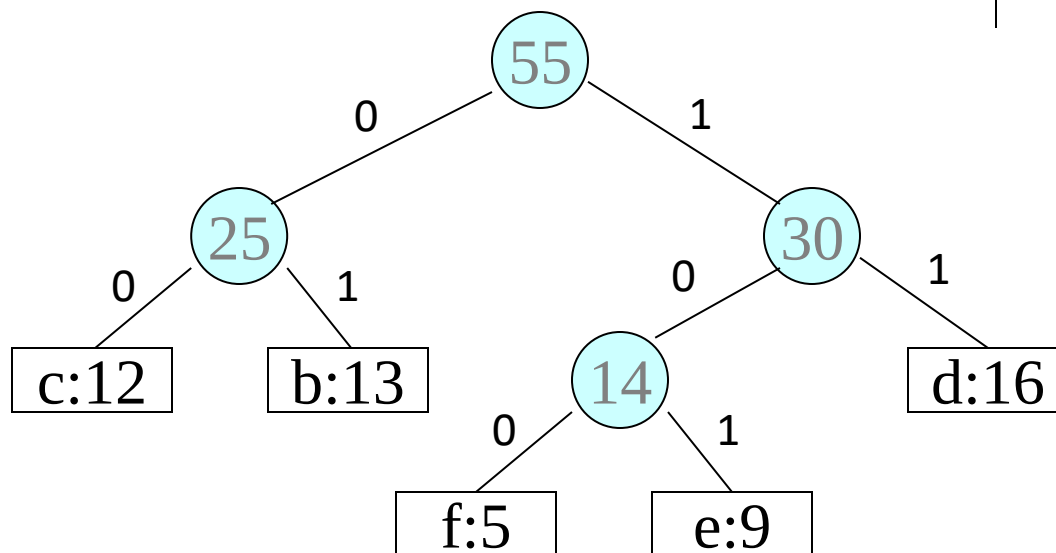
a:45



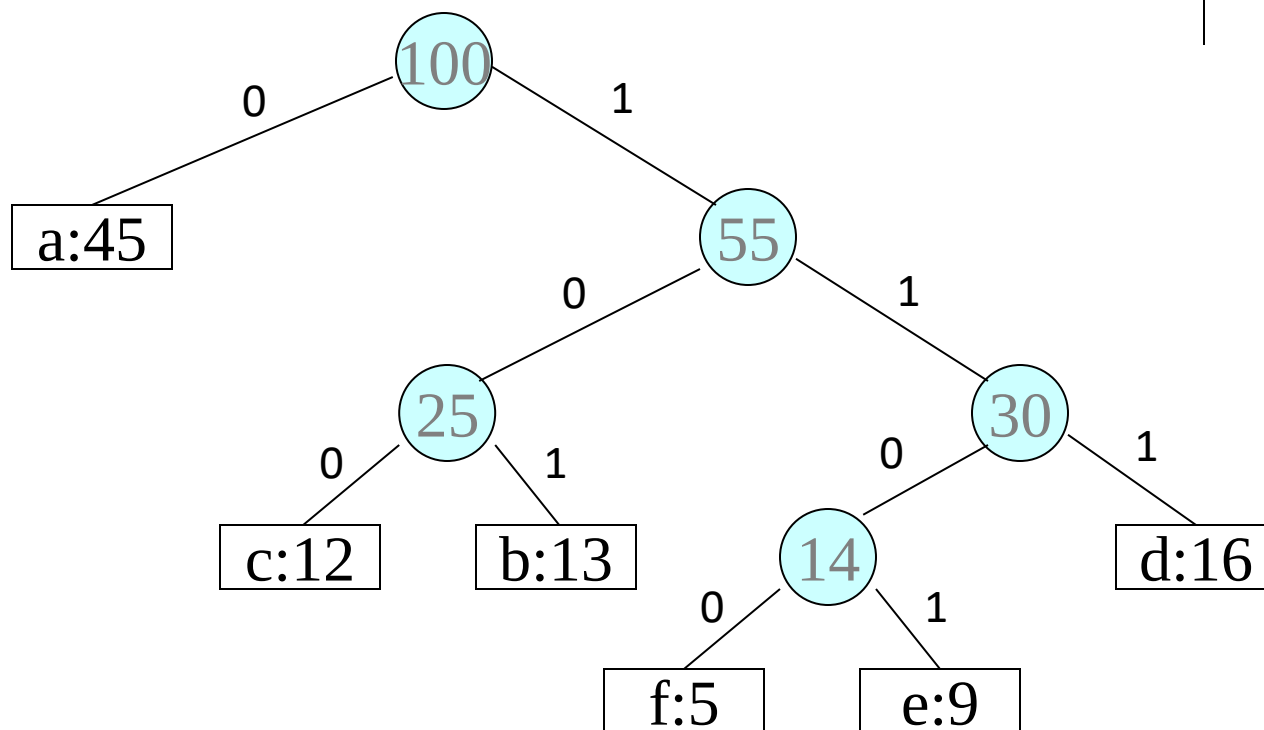
构造哈夫曼编码

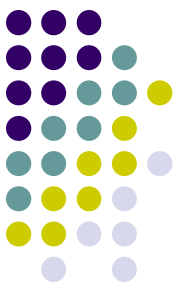


a:45



构造哈夫曼编码

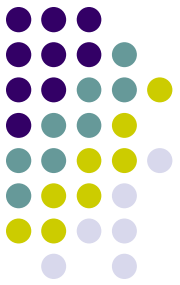




4.4 哈夫曼编码

在书上给出的算法huffmanTree中，编码字符集中每一字符 c 的频率是 $f(c)$ 。以 f 为键值的优先队列 Q 用在贪心选择时有效地确定算法当前要合并的2棵具有最小频率的树。一旦2棵具有最小频率的树合并后，产生一棵新的树，其频率为合并的2棵树的频率之和，并将新树插入优先队列 Q 。经过 $n-1$ 次的合并后，优先队列中只剩下一棵树，即所要求的树 T 。

算法huffmanTree用最小堆实现优先队列 Q 。初始化优先队列需要 $O(n)$ 计算时间，由于最小堆的removeMin和put运算均需 $O(\log n)$ 时间， $n-1$ 次的合并总共需要 $O(n \log n)$ 计算时间。因此，关于 n 个字符的哈夫曼算法的计算时间为 $O(n \log n)$ 。



4.4 哈夫曼编码

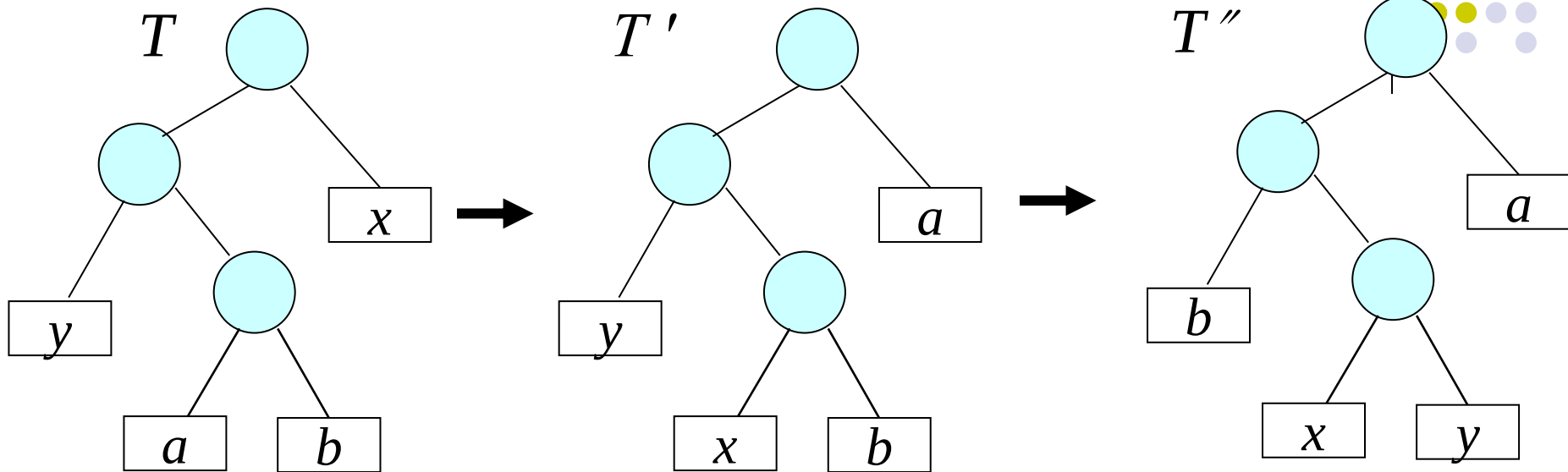
3、哈夫曼算法的正确性

要证明哈夫曼算法的正确性，只要证明最优前缀码问题具有贪心选择性质和最优子结构性质。

(1) 贪心选择性质

(2) 最优子结构性质

贪心选择性质



$$\begin{aligned}
 B(T) - B(T') &= \sum_{c \in C} f(c)d_T(c) - \sum_{c \in C} f(c)d_{T'}(c) \\
 &= f[x]d_T(x) + f[a]d_T(a) - f[x]d_{T'}(x) - f[a]d_{T'}(a) \\
 &= f[x]d_T(x) + f[a]d_T(a) - f[x]d_T(a) - f[a]d_T(x) \\
 &= (f[a] - f[x])(d_T(a) - d_T(x)) \\
 &\geq 0
 \end{aligned}$$

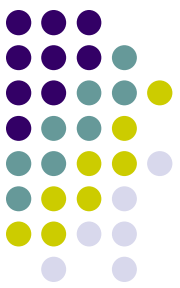


4.5 单源最短路径

给定带权有向图 $G = (V, E)$ ，其中每条边的权是非负实数。另外，还给定 V 中的一个顶点，称为源。现在要计算从源到所有其它各顶点的最短路长度。这里路的长度是指路上各边权之和。这个问题通常称为单源最短路径问题。

1、算法基本思想

Dijkstra算法是解单源最短路径问题的贪心算法。



4.5 单源最短路径

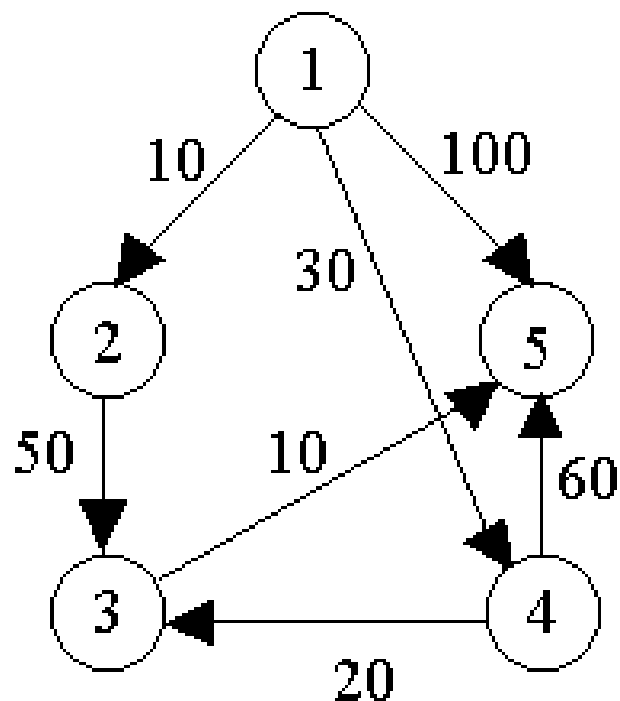
其基本思想是，设置顶点集合 S 并不断地作贪心选择来扩充这个集合。一个顶点属于集合 S 当且仅当从源到该顶点的最短路径长度已知。

初始时， S 中仅含有源。设 u 是 G 的某一个顶点，把从源到 u 且中间只经过 S 中顶点的路称为从源到 u 的特殊路径，并用数组 $dist$ 记录当前每个顶点所对应的最短特殊路径长度。Dijkstra算法每次从 $V-S$ 中取出具有最短特殊路长度的顶点 u ，将 u 添加到 S 中，同时对数组 $dist$ 作必要的修改。一旦 S 包含了所有 V 中顶点， $dist$ 就记录了从源到所有其它顶点之间的最短路径长度。



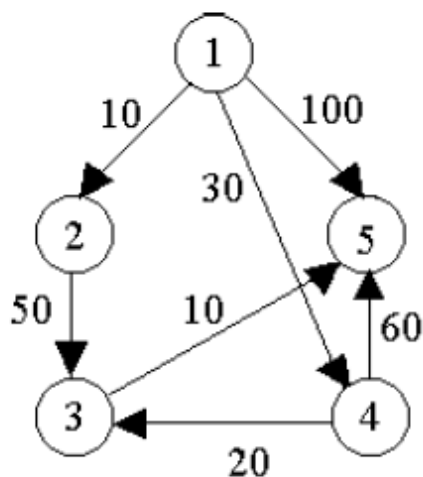
4.5 单源最短路径

例如，对右图中的有向图，应用Dijkstra算法计算从源顶点1到其它顶点间最短路径的过程列在下页的表中。





例如，对右图中的有向图，应用迪杰斯特拉算法计算从源顶点1到其它顶点间最短路径的过程列如下表所示。



迭代	S	u	dist[2]	dist[3]	dist[4]	dist[5]
初始	{1}	-	10	maxint	30	100
1	{1,2}	2	10	60	30	100
2	{1,2,4}	4	10	50	30	90
3	{1,2,4,3}	3	10	50	30	60
4	{1,2,4,3,5}	5	10	50	30	60

● 算法描述如下:

Dijkstra(int n, int v, Type dist[], int prev[], Type ** c)

{ // 单源最短路径问题的迪杰斯特拉算法,

bool s[maxint];

for (int i = 1; i <= n; i++) {

dist[i] = c[v][i];

s[i] = false;

if (dist[i] == maxint) prev[i] = 0;

else prev[i] = v;

}

dist[v] = 0; s[v] = true;

for (int i = 1; i < n; i++) {

int temp = maxint;

int u = v;

for (int j = 1; j <= n; j++)

if ((!s[j]) && (dist[j] < temp)) { u = j; temp = dist[j]; }

s[u] = true;

for (int j = 1; j <= n; j++)

if ((!s[j]) && (c[u][j] < maxint)) {

Type newdist = dist[u] + c[u][j];

if (newdist < dist[j]) { dist[j] = newdist; prev[j] = u; }

}

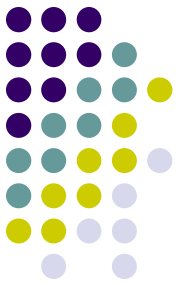
}

其中:

c[i][j] 表示边 (i,j) 的权, n 是顶点个数, v 表示源,

dist[i] 表示当前从源到顶点 i 的最短特殊路径长度

思考: 本算法只是给出了从源顶点到其他顶点间的最短路径长度, 并没有记录相应的最短路径。该如何修改才可以记录相应的最短路径?



4.5 单源最短路径

2、算法的正确性和计算复杂性

- (1) 贪心选择性质
- (2) 最优子结构性质
- (3) 计算复杂性

对于具有 n 个顶点和 e 条边的带权有向图，如果用带权邻接矩阵表示这个图，那么Dijkstra算法的主循环体需要 $O(n)$ 时间。这个循环需要执行 $n-1$ 次，所以完成循环需要 $O(n^2)$ 时间。算法的其余部分所需要时间不超过 $O(n^2)$ 。



- **贪心策略为：**从 $V-S$ 中选择具有最短特殊路径的顶点 u ，从而确定从源到 u 的最短路径长度 $\text{dist}[u]$ 。

- **贪心选择性质证明：**

证明：（反证法）

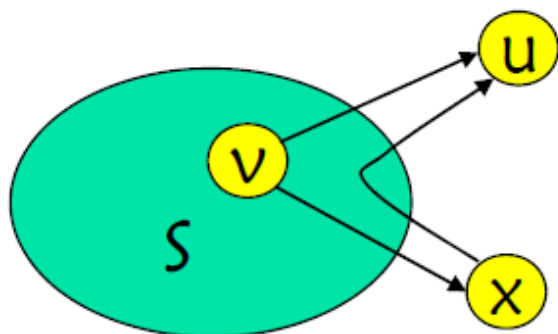
即证明从源到 u 没有更短的其他路径。

假设存在一条从源到 u 且长度比 $\text{dist}[u]$ 更短的路，设这条路初次走出 S 之外到达顶点为 $x \notin V-S$ ，然后徘徊于 S 内外若干次，最后离开达到 u ，如上图所示。

在这条路径上，分别记 $d(v,x)$ ， $d(x,u)$ 和 $d(v,u)$ 为顶点 v 到顶点 x ，顶点 x 到顶点 u 和顶点 v 到顶点 u 的路长，那么

$$\text{dist}[x] \leq d(v,x) + d(x,u) = d(v,u) < \text{dist}[u]$$

利用边权的非负性，可知 $d(x,u) \geq 0$ ，从而推得 $\text{dist}[x] < \text{dist}[u]$ 。此为矛盾。这就证明了 $\text{dist}[u]$ 是从源到顶点 u 的最短路径长度。

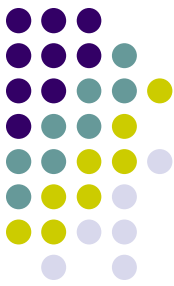




4.6 最小生成树

设 $G = (V, E)$ 是无向连通带权图，即一个网络。E中每条边 (v, w) 的权为 $c[v][w]$ 。如果G的子图 G' 是一棵包含G的所有顶点的树，则称 G' 为G的生成树。生成树上各边权的总和称为该生成树的耗费。在G的所有生成树中，耗费最小的生成树称为G的最小生成树。

网络的最小生成树在实际中有广泛应用。例如，在设计通信网络时，用图的顶点表示城市，用边 (v, w) 的权 $c[v][w]$ 表示建立城市v和城市w之间的通信线路所需的费用，则最小生成树就给出了建立通信网络的最经济的方案。



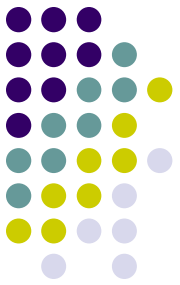
4.6 最小生成树

1、最小生成树性质

用贪心算法设计策略可以设计出构造最小生成树的有效算法。本节介绍的构造最小生成树的Prim算法和Kruskal算法都可以看作是应用贪心算法设计策略的例子。尽管这2个算法做贪心选择的方式不同，它们都利用了下面的最小生成树性质：

设 $G=(V, E)$ 是连通带权图， U 是 V 的真子集。如果 $(u, v) \in E$ ，且 $u \in U$ ， $v \in V-U$ ，且在所有这样的边中， (u, v) 的权 $c[u][v]$ 最小，那么一定存在 G 的一棵最小生成树，它以 (u, v) 为其中一条边。这个性质有时也称为MST性质。

证明MST性质。P125



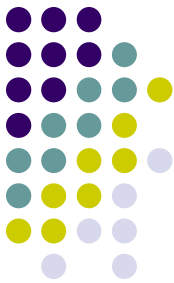
4.6 最小生成树

2、Prim算法

设 $G=(V, E)$ 是连通带权图， $V=\{1, 2, \dots, n\}$ 。

构造 G 的最小生成树的Prim算法的基本思想是：首先置 $S=\{1\}$ ，然后，只要 S 是 V 的真子集，就作如下的贪心选择：选取满足条件 $i \in S$ ， $j \in V-S$ ，且 $c[i][j]$ 最小的边，将顶点 j 添加到 S 中。这个过程一直进行到 $S=V$ 时为止。

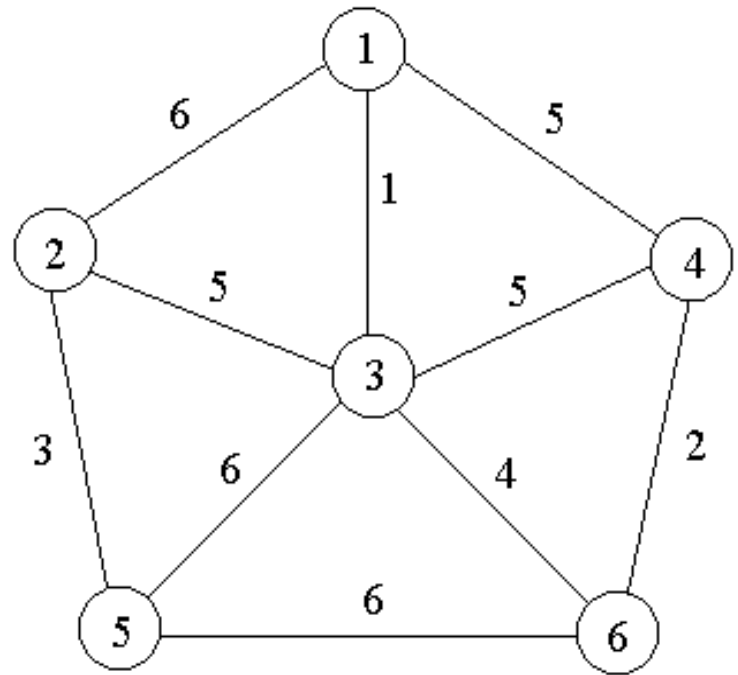
在这个过程中选取到的所有边恰好构成 G 的一棵最小生成树。



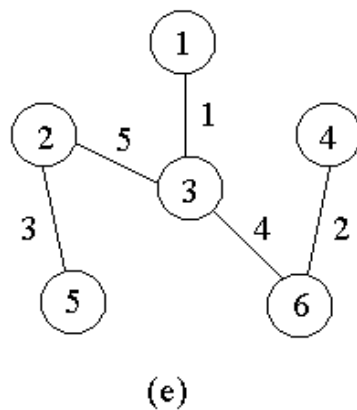
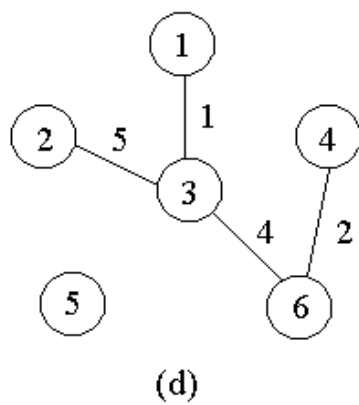
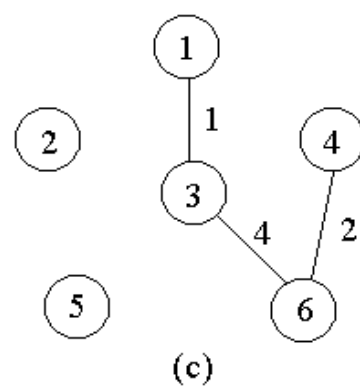
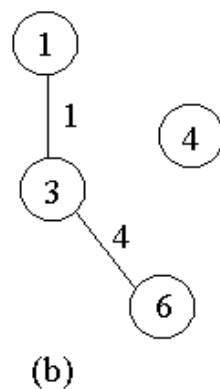
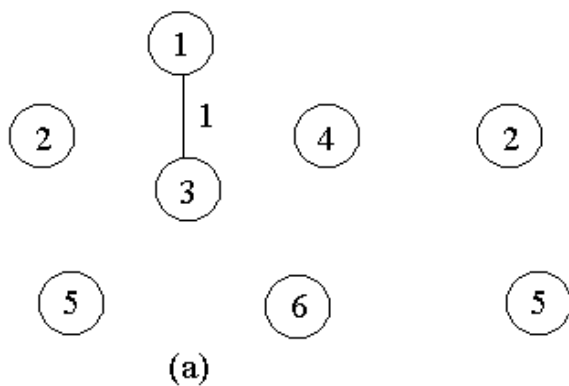
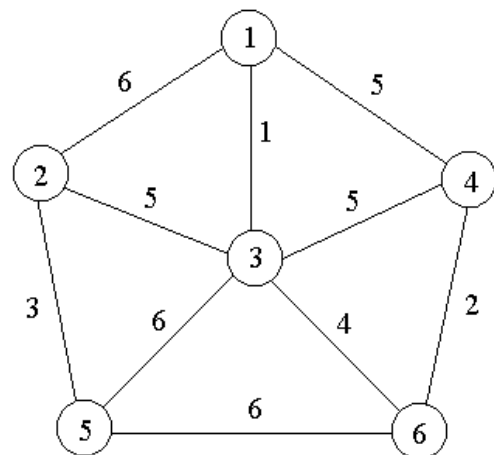
4.6 最小生成树

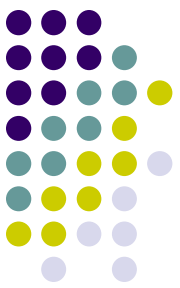
利用最小生成树性质和数学归纳法容易证明，上述算法中的边集合 T 始终包含 G 的某棵最小生成树中的边。因此，在算法结束时， T 中的所有边构成 G 的一棵最小生成树。

例如，对于右图中的带权图，按Prim算法选取边的过程如下页图所示。



4.6 最小生成树



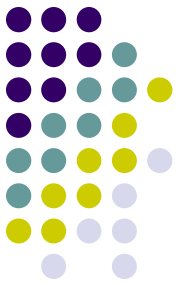


4.6 最小生成树

在上述Prim算法中，还应当考虑如何有效地找出满足条件 $i \in S, j \in V-S$ ，且权 $c[i][j]$ 最小的边 (i, j) 。实现这个目的的较简单的办法是设置2个数组closest和lowcost。

在Prim算法执行过程中，先找出 $V-S$ 中使lowcost值最小的顶点 j ，然后根据数组closest选取边 $(j, \text{closest}[j])$ ，最后将 j 添加到 S 中，并对closest和lowcost作必要的修改。

用这个办法实现的Prim算法所需的计算时间为 $O(n^2)$



4.6 最小生成树

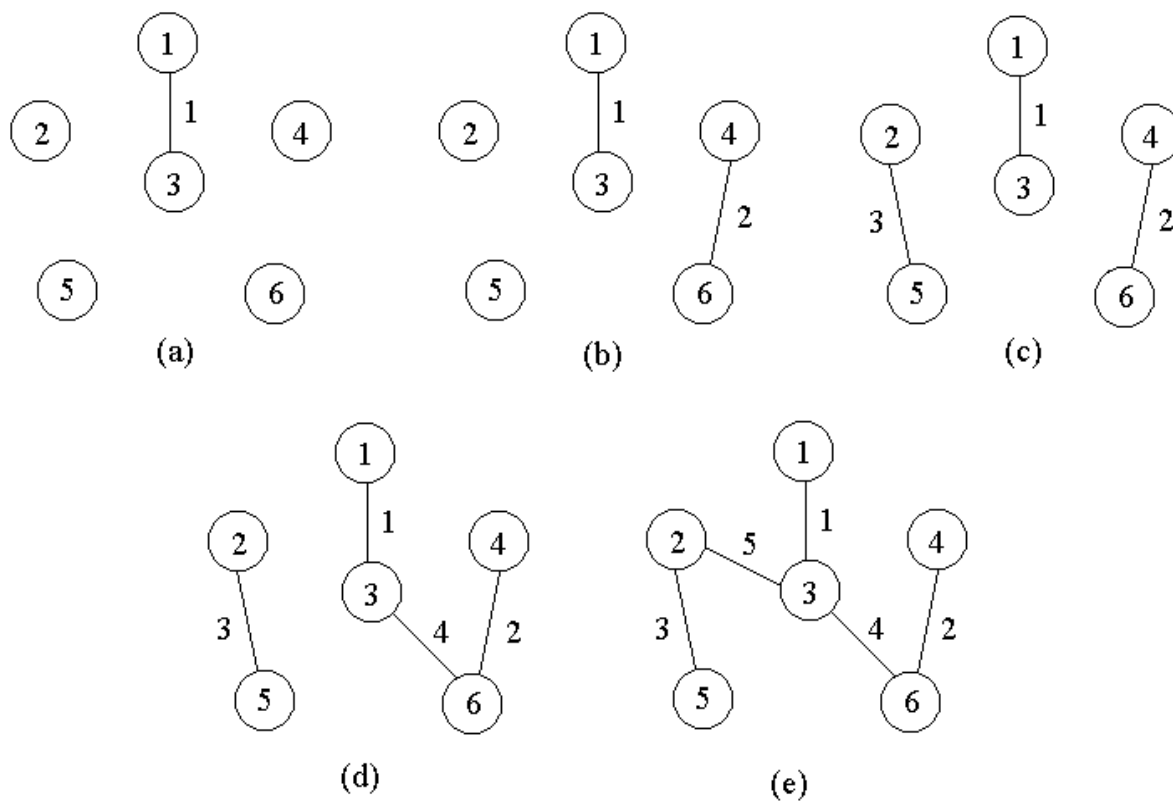
3、Kruskal算法

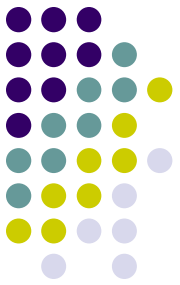
Kruskal算法构造G的最小生成树的基本思想是，首先将G的 n 个顶点看成 n 个孤立的连通分支。将所有的边按权从小到大排序。然后从第一条边开始，依边权递增的顺序查看每一条边，并按下述方法连接2个不同的连通分支：当查看到第 k 条边 (v, w) 时，如果端点 v 和 w 分别是当前2个不同的连通分支 T_1 和 T_2 中的顶点时，就用边 (v, w) 将 T_1 和 T_2 连接成一个连通分支，然后继续查看第 $k+1$ 条边；如果端点 v 和 w 在当前的同一个连通分支中，就直接再查看第 $k+1$ 条边。这个过程一直进行到只剩下一个连通分支时为止。



4.6 最小生成树

例如，对前面的连通带权图，按Kruskal算法顺序得到的最小生成树上的边如下图所示。





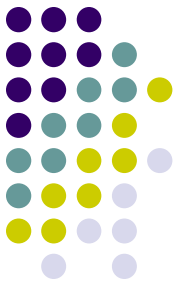
4.6 最小生成树

关于集合的一些基本运算可用于实现Kruskal算法。

按权的递增顺序查看等价于对优先队列执行removeMin运算。可以用堆实现这个优先队列。

对一个由连通分支组成的集合不断进行修改，需要用到抽象数据类型并查集UnionFind所支持的基本运算。

当图的边数为 e 时，Kruskal算法所需的计算时间是 $O(e \log e)$ 。当 $e = \Omega(n^2)$ 时，Kruskal算法比Prim算法差，但当 $e = o(n^2)$ 时，Kruskal算法却比Prim算法好得多。

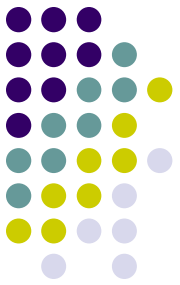


4.7 多机调度问题

多机调度问题要求给出一种作业调度方案，使所给的 n 个作业在尽可能短的时间内由 m 台机器加工处理完成。

约定，每个作业均可在任何一台机器上加工处理，但未完工前不允许中断处理。作业不能拆分成更小的子作业。

解法。对于这一问题，用贪心选择策略有时可以设计出较好的近似算法。

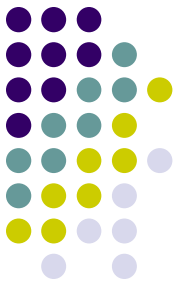


4.7 多机调度问题

采用最长处理时间作业优先的贪心选择策略可以设计出解多机调度问题的较好的近似算法。

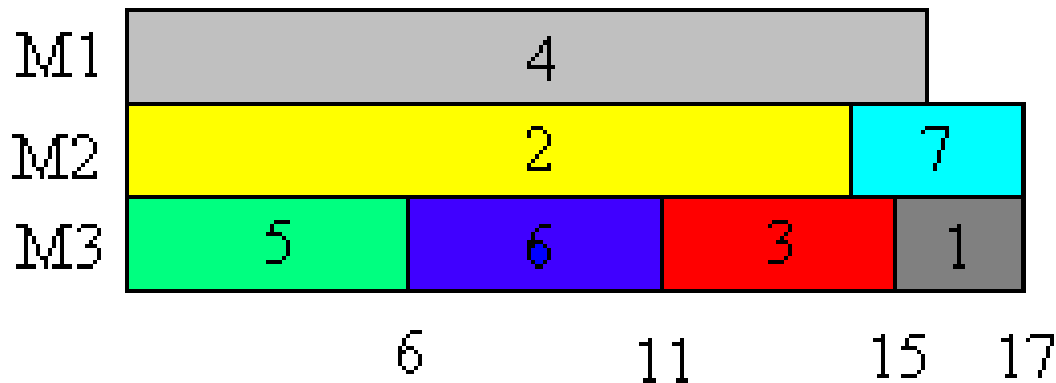
按此策略，当 $n \leq m$ 时，只要将机器 i 的 $[0, t_i]$ 时间区间分配给作业 i 即可，算法只需要 $O(1)$ 时间。

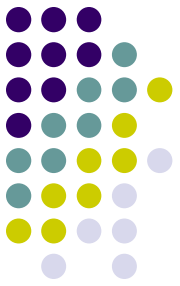
当 $n > m$ 时，首先将 n 个作业依其所需的处理时间从大到小排序。然后依此顺序将作业分配给空闲的处理机。算法所需的计算时间为 $O(n \log n)$ 。



4.7 多机调度问题

例如，设7个独立作业{1, 2, 3, 4, 5, 6, 7}由3台机器M1, M2和M3加工处理。各作业所需的处理时间分别为{2, 14, 4, 16, 6, 5, 3}。按算法greedy产生的作业调度如下图所示，所需的加工时间为17。

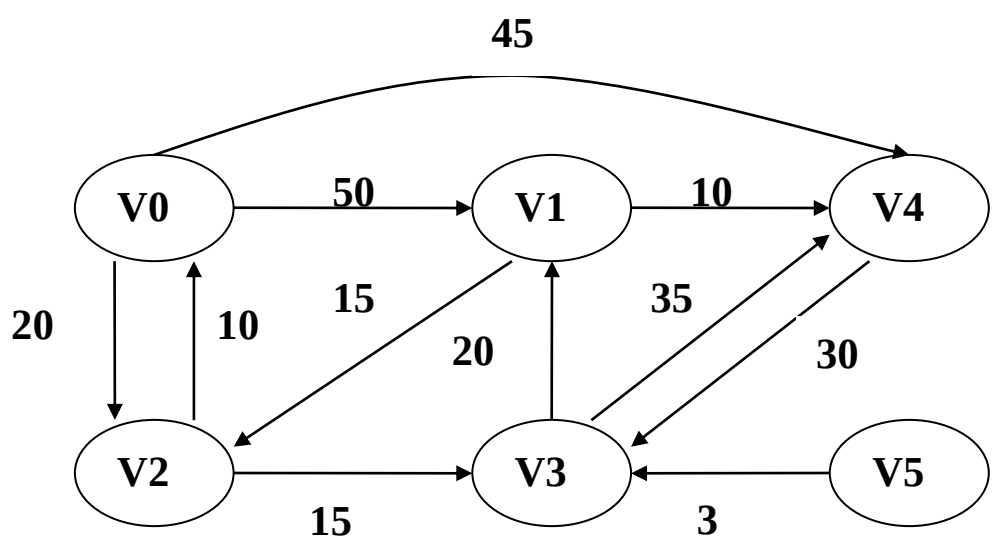




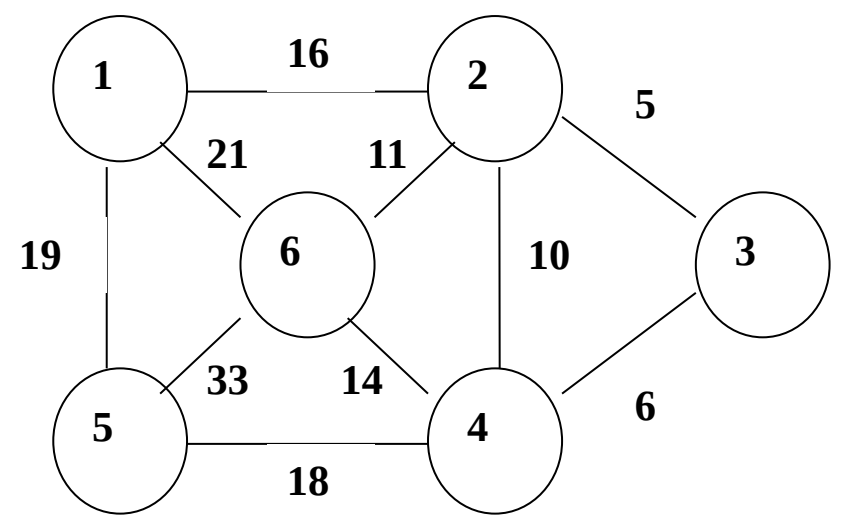
本周任务

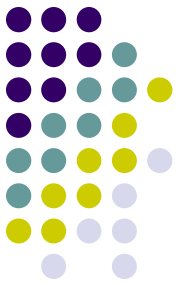
1. 证明：一颗不满的二叉树不可能与一种最优前缀编码对应。
2. 4-5, 4-16, 4-19, 4-26

● 求v0出发的 最短路径



● 求最小生成树





本周任务

- 与背包问题相关:
 1. 4-2
 2. 4-3
 3. 如何修改背包问题的动态规划算法,使得它只需要 $\Theta(C)$ 空间,其中 C 为背包容量.
 4. 定义0 / 1 / 2背包问题为: $\max\{\sum v_i x_i\}$,约束条件为: $\sum w_i x_i \leq c$ 且 $x_i \in \{0, 1, 2\}$, $1 \leq i \leq n$ 。设 P 的定义同0 / 1背包问题中的定义。
 1. 1) 从0 / 1 / 2背包问题中推出类似于0/1背包的递归公式。
 2. 2) 假设 w 为整数。编写一个类似于0/1背包的程序来计算二维数组 P , 然后确定最优分配的 x 值。
 3. 3) 分析上述算法的时间复杂度。